IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts, Science, Commerce and Management

Project – High Level Design

on

Cloud Forensic Automation For Healthcare Breaches

Student Name: Sreelakshmi Sreejith

Industry Mentor: Mr.Shashank | Guided by: Ms Divya Shetty

Table of Contents

1. Introduction 1
   1.1 Scope of the Document 1
   1.2 Intended Audience 1,2
   1.3 System Overview 3,4
2. System Design 5
   2.1 Application Design 5
   2.2 Process Flow 6
   2.3 Information Flow 7
   2.4 Components Design 8
   2.5 Key Design Considerations 9
   2.6 API Catalogue 10
3. Data Design 11
   3.1 Data Model 11
   3.2 Data Access Mechanism 12
   3.3 Data Retention Policies 12
   3.4 Data Migration 13
4. Interfaces 13
5. State and Session Management 14
6. Caching 14
7. Non-Functional Requirements 15
   7.1 Security Aspects 15
   7.2 Performance Aspects 16
8. References 17

1. Introduction

MedTrace Forensics is a web-based Cyber Forensics Command Center designed to investigate healthcare data breaches. The system enables digital forensic investigators to scan healthcare data directories, classify files based on risk level, verify file integrity using SHA-256 cryptographic hashing, generate evidence reports, and monitor threats in real time. The platform was developed in response to the HC-2026-009 case involving the MedTrace Healthcare Cloud Storage System.

With the increasing use of cloud storage in healthcare systems, ensuring data security and integrity has become critical. Sensitive data such as patient records, billing information, and medical reports must be protected from unauthorized access and tampering. This system helps in identifying suspicious file activities and ensures that any modification after a breach event is properly detected and analyzed.

The platform provides a simple and efficient interface for investigators to perform forensic analysis, visualize results, and generate reports, making the investigation process faster and more reliable.

1.1 Scope of the Document

This High-Level Design (HLD) document describes the architectural blueprint of the MedTrace Forensics system. It covers the system's structural design, component interactions, data flow, API contracts, data model, interface definitions, security mechanisms, and non-functional requirements. This document serves as the primary design reference for developers, architects, and stakeholders involved in the development and review of the system.

1.2 Intended Audience

This document is intended for the followin stakeholders:

- Software architects and developers responsible for implementation
- Quality assurance and testing teams
- Industry mentors and academic faculty reviewers (IBM / KTU)
- Security analysts and forensic investigators
- Project evaluators and academic committee members

USE CASE DIAGRAM

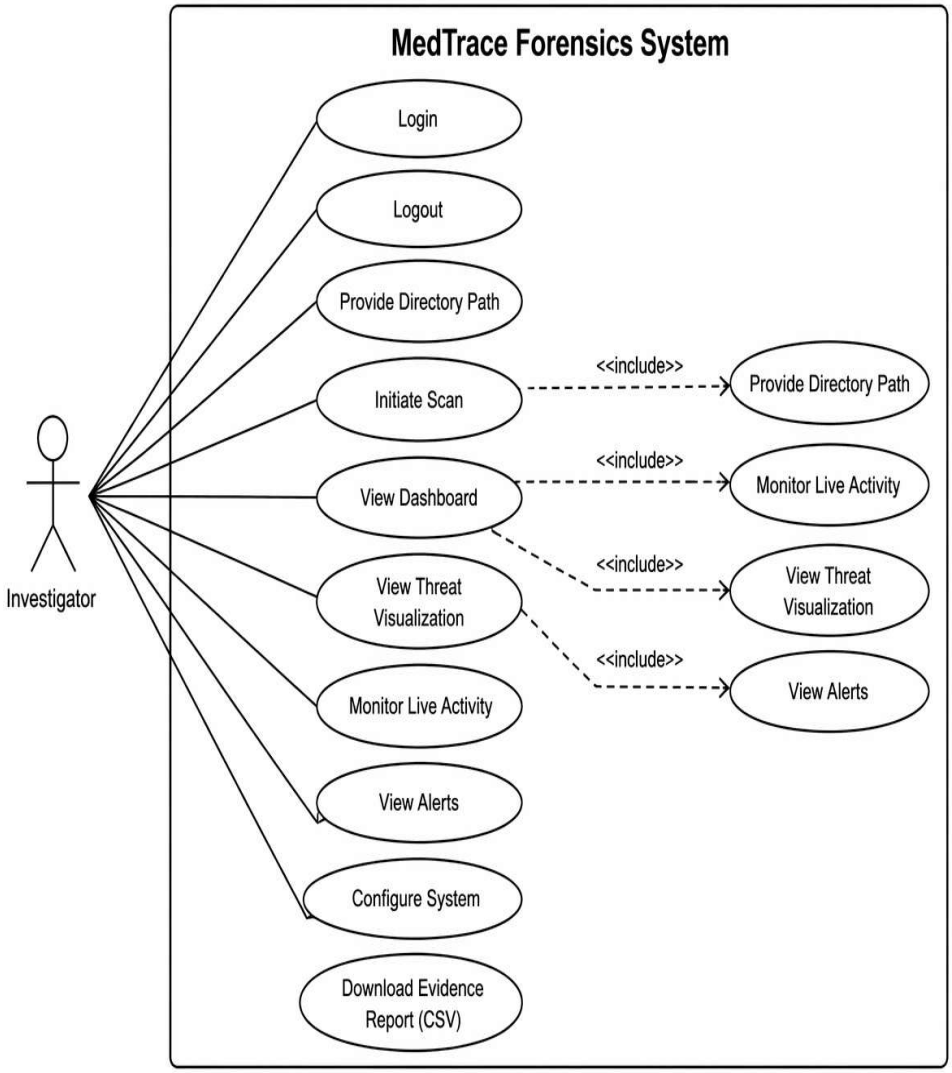


Figure 1.1: Use Case Diagram – MedTrace Forensics System

The Use Case Diagram represents the interaction between the investigator and the MedTrace Forensics system. It outlines key functionalities such as authentication, initiating scans, monitoring activities, visualizing threats, managing system settings, and generating evidence reports. This diagram provides a high-level view of user interactions within the system.

2. System Overview

MedTrace Forensics is a full-stack Flask web application with a cyberpunk-themed investigative dashboard. The system is deployed as a local server and accessed through a web browser. It provides an interactive environment for conducting forensic analysis on healthcare data stored in cloud-based systems.

The application consists of five core operational modules:

- **Command Center** – Used to initiate scans and manage the evidence workspace
- **Threat Visualizer** – Displays risk distribution through charts and structured evidence tables
- **Live Monitor** – Provides a real-time terminal view of system activity and scan logs
- **Alert Zone** – Highlights high-risk threats and suspicious file activities
- **Control Panel** – Allows system configuration and displays diagnostics

The system scans healthcare data directories such as Electronic Health Records (EHR), CSV billing datasets, and patient case summaries. Each file artifact is analyzed and classified as **High Risk** or **Low Risk** based on its modification timestamp relative to the breach date (April 8, 2026 – Case HC-2026-009).

To ensure data integrity and maintain forensic validity, the system generates **SHA-256 cryptographic hashes** for every file. This guarantees that any tampering or unauthorized modification can be detected during investigation. After completing the scan, the system automatically generates a **CSV-based evidence report** containing all analyzed file details, including metadata, hash values, risk level, and classification.

The overall system is designed to provide a structured and efficient approach for investigators to detect, analyze, and document potential data breaches in healthcare environments.

The system also provides real-time feedback to users during scanning, improving transparency of the investigation process. Its modular design allows easy integration of additional forensic features in the future. Overall, it ensures a reliable and user-friendly environment for effective forensic analysis.

SYSTEM ARCHITECTURE DIAGRAM

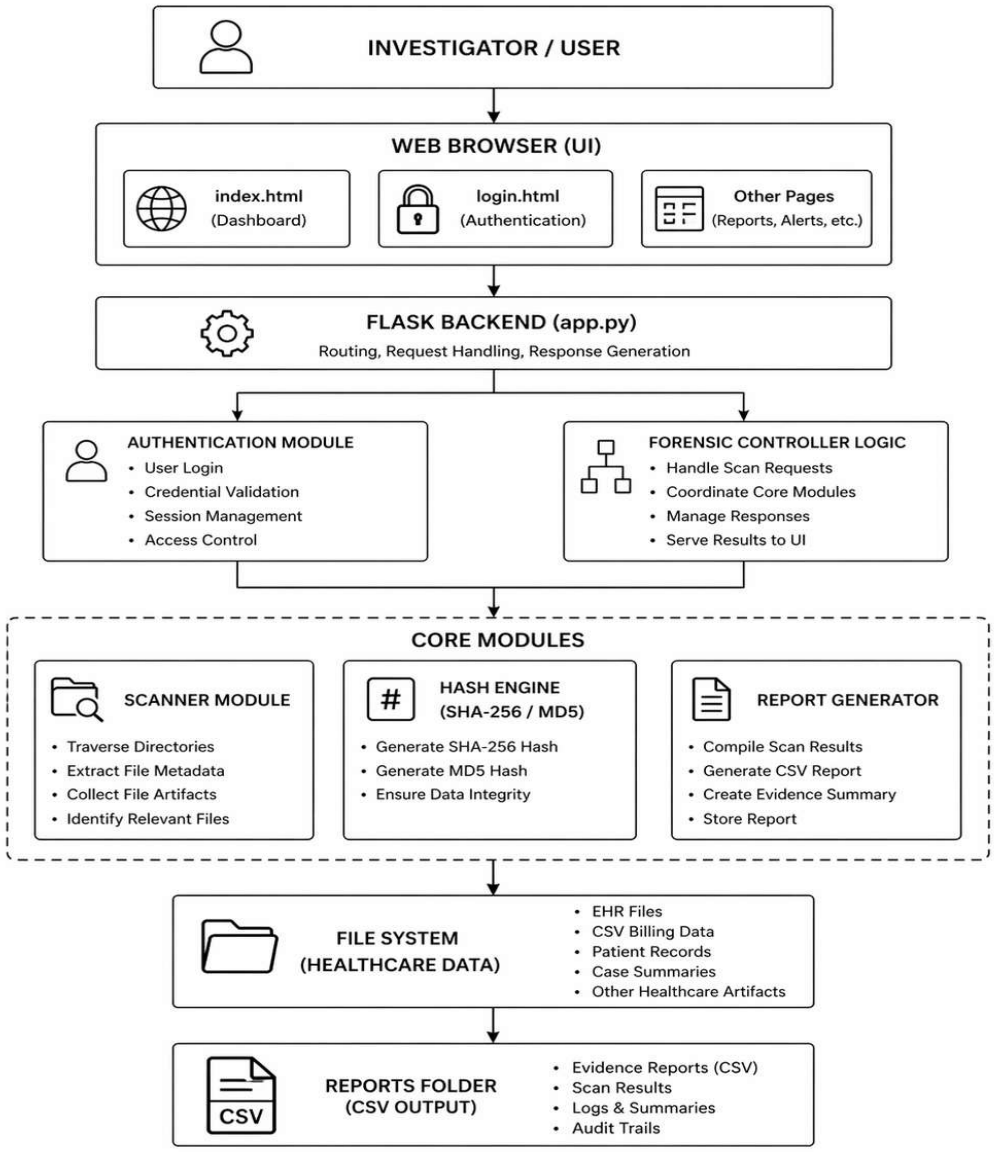


Figure 1: System Architecture Diagram

TheSystem Architecture Diagram shows the overall structure of the MedTrace Forensics system. The investigator interacts through a web browser, which communicates with the Flask backend. The backend handles authentication and coordinates core modules such as the scanner, hash engine, and report generator. These modules process healthcare data from the file system and generate CSV evidence reports, ensuring a modular and efficient design.

3. System Design

3.1 Application Design

The MedTrace Forensics system follows the **Model-View-Controller (MVC)** architectural pattern implemented using Flask. This design ensures separation of concerns, making the system modular, scalable, and easy to maintain.

The **backend (app.py)** acts as the controller, handling HTTP requests, managing routing, and coordinating between different modules. It processes user inputs such as scan requests and interacts with the forensic modules to generate results.

The **frontend components** such as *index.html* and *login.html* act as the view layer. These templates are rendered using Jinja2 and provide an interactive user interface for investigators. The frontend is enhanced using CSS and JavaScript to display charts, logs, and real-time updates.

The **model layer** consists of the scanner and hash engine modules. These modules are responsible for processing file system artifacts, extracting metadata, generating hash values, and returning structured JSON responses to the controller.

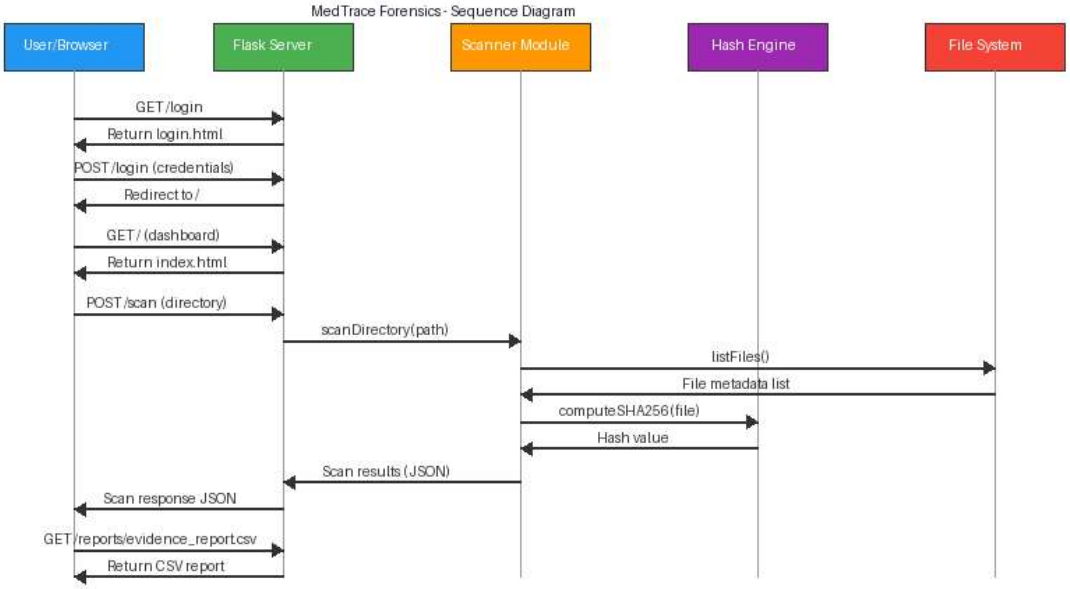
The layered architecture of the system is shown below:

Layer	Component	Technology
Presentation	index.html, login.html, style.css, script.js	HTML5, CSS3, JavaScript, Chart.js
Application	app.py (Flask routes), modules/	Python 3, Flask 3.0+
Business Logic	Scanner Module, Hash Engine	Python (hashlib, os, datetime)
Data	sample_healthcare_data/, reports/	CSV, TXT, File System

This layered design improves system maintainability by clearly separating user interface, processing logic, and data handling components. It also allows independent updates to each layer without affecting the overall system functionality.

3.2 Process Flow

The following sequence diagram illustrates the interaction between the user, Flask server, scanner module, hash engine, and the file system during a forensic scan operation:



The process begins when the user logs into the system and provides the directory path containing healthcare data. Once the scan is initiated, the request is sent to the Flask backend, which acts as the controller.

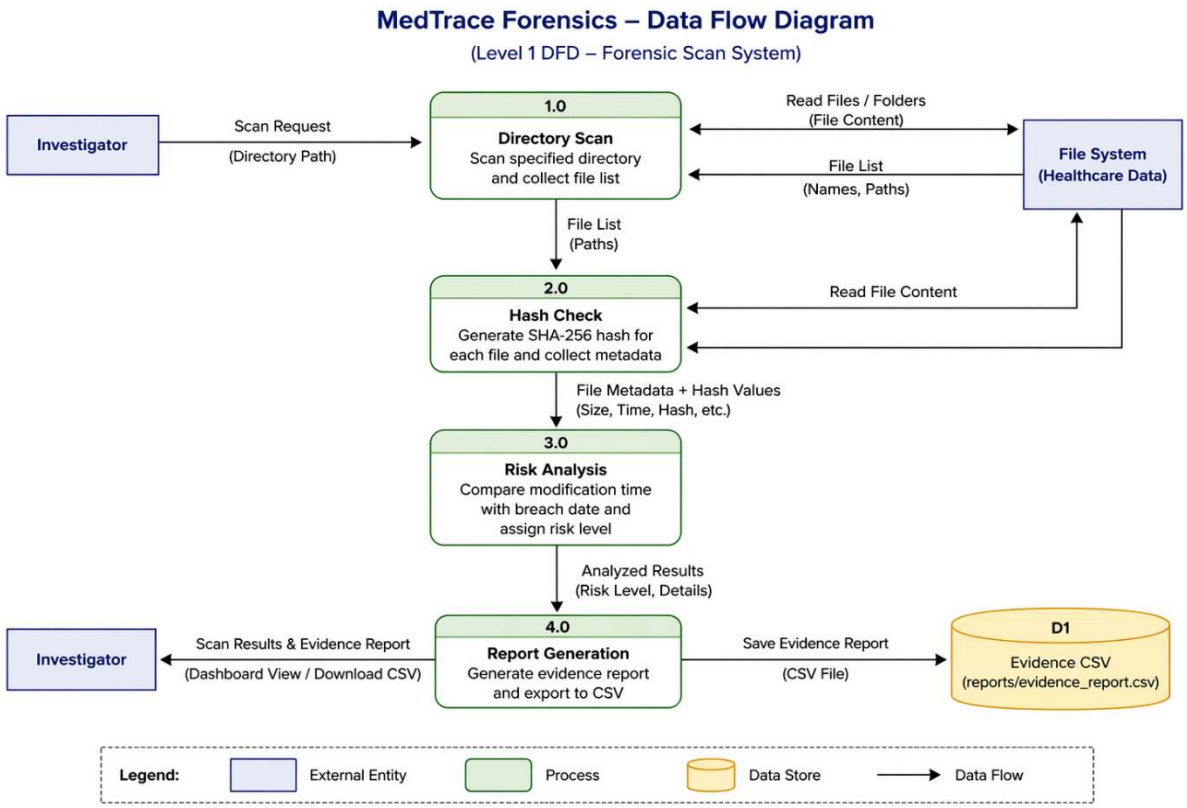
The backend then invokes the scanner module to traverse the specified directory and identify valid files. For each file, metadata such as file name, size, timestamps, and permissions is extracted. The hash engine is then used to generate SHA-256 and MD5 hash values to ensure file integrity.

After processing, the system compares the file modification time with the defined breach date to classify each file as High Risk or Low Risk. The processed data is then returned to the backend, which sends the results to the frontend for visualization.

Finally, a structured CSV evidence report is generated and made available for download by the investigator.

3.3 Information Flow

The data flow diagram below represents how data moves between external entities (investigators and file system) and internal processing units within the MedTrace Forensics system.



The process begins when the investigator provides a directory path as input. This request is handled by the Flask backend, which coordinates the forensic processing workflow. The scanner module accesses the file system and retrieves all relevant files while extracting metadata such as file size, timestamps, and permissions.

The extracted information is passed to the hash engine, where cryptographic hash values are generated to ensure file integrity. The system then performs risk classification by comparing file modification times with the defined breach date.

Finally, the processed data is returned to the backend and displayed on the dashboard. A structured CSV evidence report is also generated for download and further analysis.

3.4 Components Design

The MedTrace Forensics system is composed of five major components, each responsible for a distinct aspect of the forensic workflow. These components work together to perform efficient scanning, analysis, and reporting of healthcare data.

Component	Responsibility	Key Functions
Authentication Module	Login/logout, session management	Validate credentials, create Flask session, redirect to dashboard
Forensic Scanner	Directory traversal, metadata extraction	os.walk(), file stat, breach timestamp comparison, risk scoring
SHA-256 Hash Engine	File integrity verification	hashlib.sha256(), chunked file reading, hash comparison
Report Generator	CSV evidence export	Write scan results to /reports/evidence_report.csv
Dashboard UI	Real-time visualization	Chart.js charts, live terminal, alert cards, control panel features

Each component is designed to handle a specific function within the system, ensuring a clear separation of responsibilities. The **Authentication Module** secures access to the system, allowing only authorized users to perform forensic operations.

The **Forensic Scanner** acts as the core processing unit, traversing directories and collecting file-related information. The **Hash Engine** ensures data integrity by generating cryptographic hashes, which help in detecting any unauthorized modifications.

The **Report Generator** compiles all processed data into a structured CSV format, making it easy for investigators to analyze and document findings. The **Dashboard UI** provides an interactive interface for visualizing results, monitoring activities, and managing the investigation workflow.The modular design of the system improves maintainability and scalability. It allows individual components to be updated or enhanced without affecting other parts of the system, making it flexible for future improvements.

4. Key Design Considerations

The design of the MedTrace Forensics system is guided by several important considerations to ensure security, reliability, and scalability.

It focuses on maintaining data integrity, ensuring accurate forensic analysis, and providing a secure environment for handling sensitive healthcare information. The system is designed to support efficient investigation workflows while maintaining simplicity and ease of use.

• **Authentication-first access:** All dashboard routes are protected. Unauthenticated users are redirected to the login page, ensuring secure system access.

• **Chain of custody:** SHA-256 hashes are generated for every scanned file, ensuring that evidence integrity is maintained and can be verified at any stage.

• **Temporal risk classification:** Files modified after the breach date (April 8, 2026) are classified as High Risk, while others are marked as Low Risk, enabling focused investigation.

• **Stateless scan model:** Each scan is independent and initiated through a request. Results are processed in-memory and exported as CSV without using a database.

• **Frontend-backend separation:** The backend handles processing and routing, while the frontend manages visualization, charts, and user interaction.

• **Modular backend design:** Core functionalities such as scanning and hashing are separated into modules, allowing easy maintenance and future upgrades.

• **Healthcare data sensitivity:** The system processes sensitive healthcare datasets such as EHR and billing data, ensuring confidentiality and responsible data handling.

• **Scalability consideration:** The modular architecture allows the system to handle larger datasets in the future. Additional features such as parallel processing and database integration can be added without major changes.

• **Usability and visualization:** The system provides a user-friendly dashboard with charts and alerts, making it easier for investigators to understand results and make decisions quickly.

5. API Catalogue

The following APIs are used for communication between the frontend and backend components of the system:

Endpoint	Method	Auth	Description	Response
/login	GET	None	Render login form	HTML (login.html)
/login	POST	None	Authenticate user credentials	Redirect / or 401
/logout	GET	Session	Clear session, logout	Redirect /login
/	GET	Session	Render forensics dashboard	HTML (index.html)
/scan	POST	Session	Initiate directory scan, return JSON results	JSON {files, stats}
/reports/evidence_report.csv	GET	Session	Download generated evidence CSV	File (CSV)

These APIs enable smooth interaction between the user interface and backend processing, ensuring efficient execution of forensic operations and data retrieval.

In addition to basic request handling, these APIs ensure secure communication by validating user sessions before allowing access to protected resources. This prevents unauthorized operations such as scanning directories or downloading reports.

The APIs are designed to be lightweight and efficient, enabling quick data exchange between the frontend and backend. The use of JSON responses allows easy integration with the user interface for real-time updates and visualization.

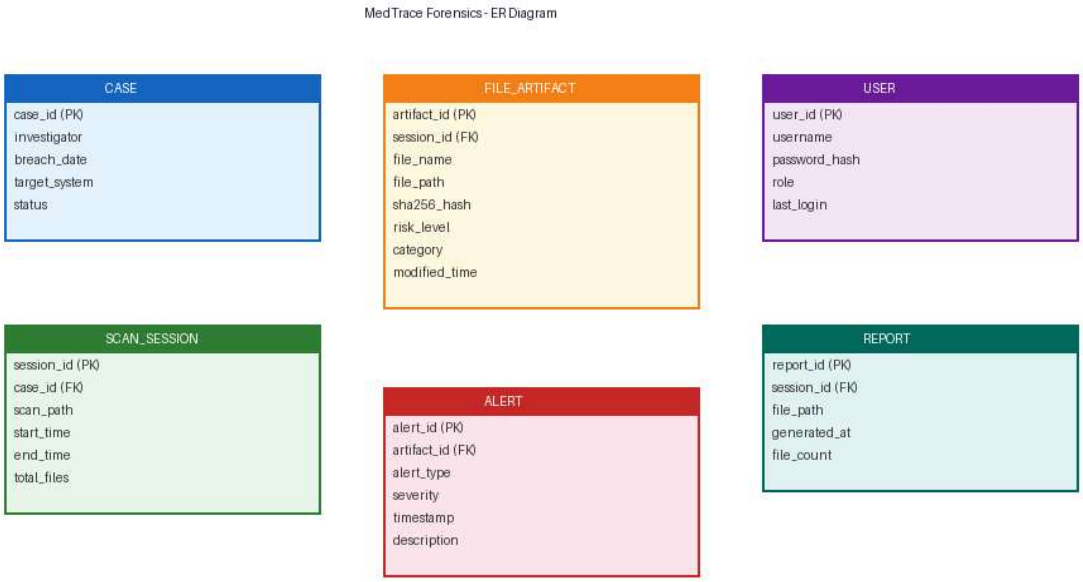
Furthermore, the API structure is modular, allowing future enhancements such as additional endpoints for advanced analytics, report history, or integration with external forensic tools.

6. Data Design

6.1 Data Model

MedTrace Forensics does not use a relational database in the current implementation. Instead, data is managed using the file system and in-memory Python data structures. This approach keeps the system lightweight and suitable for fast forensic analysis without database overhead.

The following conceptual model represents the logical data entities used within the system:



The ER diagram represents the relationship between different entities involved in the forensic process. It shows how file artifacts are linked to scan sessions and associated with specific cases. Each entity captures important attributes required for analysis and reporting. This structure helps in organizing data efficiently and supports smooth forensic investigation.

Additionally, the relationships between entities help in tracking the flow of data during each scan operation. It ensures that every file artifact can be traced back to its corresponding scan session and case. This structured representation improves data consistency and makes the system easier to understand and maintain. It also supports efficient report generation and future enhancements.

It also helps in clearly identifying how data flows between different components of the system. The structured design improves traceability of evidence during investigations. This makes the system more reliable and easier to analyze.

Key data entities and their attributes:

Entity	Key Attributes	Storage Medium
CASE	case_id, investigator, breach_date, target_system	Hardcoded in HTML (HC-2026-009)
FILE_ARTIFACT	file_name, path, sha256_hash, risk_level, category, modified_time	In-memory (Python list) + CSV export
SCAN_SESSION	scan_path, start_time, end_time, total_files, high_risk, low_risk	In-memory (Python dict)
ALERT	artifact_id, alert_type, severity, timestamp	In-memory (JavaScript array, client side)
USER	username, password_hash	Hardcoded / config file
REPORT	file_path, generated_at, file_count	File system (/reports/)

6.2 Data Access Mechanism

The system accesses data directly from the file system using Python's built-in modules. The scanner module uses the os.walk() function to traverse directories and identify relevant files, while os.stat() is used to extract metadata such as file size and timestamps.

The hash engine reads files in binary mode using chunk-based reading (read(65536)), ensuring efficient handling of large files without excessive memory usage. The processed data is then structured and returned to the backend.

The generated evidence report is written using Python's csv module and is made available for download through Flask routes using send_file().

6.3 Data Retention Policies

Scan results are stored in memory for the duration of the application session. Once the scan is completed, a CSV report (evidence_report.csv) is generated and stored in the /reports/ directory.

The report persists on disk until it is overwritten by a new scan. The system does not store sensitive healthcare data such as patient details; instead, it retains only file metadata, hash values, timestamps, and risk classifications, ensuring minimal data exposure.

6.4 Data Migration

In the current implementation, no data migration mechanism is required since the system relies on file-based storage. However, for future scalability, the system can be extended to use databases such as SQLite or PostgreSQL.

A migration process can be implemented to convert existing CSV evidence reports into structured database records. This would enable features such as historical analysis, indexing, and advanced querying based on case ID and scan timestamps.

7. Interfaces

MedTrace Forensics provides multiple interfaces for interaction between users, system components, and external resources. These interfaces ensure smooth communication and efficient system operation.

Interface Type	Description	Technology
User Interface	Cyber-themed browser dashboard with 5 tabs (Command, Threats, Monitor, Alerts, Control)	HTML5, CSS3, JavaScript, Chart.js
REST API	JSON-based scan API, login/logout routes, CSV report download	Flask HTTP routes
File System Interface	Directory scanner reads target path recursively	Python os module
Export Interface	CSV evidence report downloadable by investigator	Flask send_file / static route

- The User Interface allows investigators to perform scans, view results, and monitor activity in real time.
- The REST API enables communication between frontend and backend for login, scanning, and report retrieval.
- The File System Interface handles access and analysis of files in directories.
- The Export Interface allows downloading structured reports for documentation.

8. State and Session Management

The MedTrace Forensics system uses Flask's built-in session mechanism to manage user authentication and access control. When a user successfully logs into the system, a session variable (logged_in = True) is created and stored securely using a server-side secret key. This session variable is checked for every protected route to ensure that only authorized users can access the dashboard and perform forensic operations.

If a user attempts to access restricted pages without authentication, they are automatically redirected to the login page. The session remains active as long as the browser session is open and is terminated when the user logs out using the logout option or closes the browser. This ensures basic security and controlled access to the system.

In addition to authentication, scan-related state is managed on the client side using JavaScript. The system dynamically updates the scan progress, status indicators (such as IDLE, SCANNING, and COMPLETE), and result data using asynchronous API calls. The live terminal view is simulated by progressively displaying logs during the scanning process, giving a real-time monitoring experience to the investigator.

9. Caching

The current implementation of the MedTrace Forensics system does not include a dedicated caching mechanism. All operations, including file scanning and hash generation, are performed in real time to ensure accuracy and reliability of forensic results.

Static assets such as CSS and JavaScript files are served directly by the Flask server and may be cached by the browser using standard HTTP caching mechanisms. However, critical operations like SHA-256 hash generation are executed freshly during each scan to maintain evidence integrity and prevent the use of outdated data.

While caching can improve performance, especially when scanning large datasets repeatedly, it is not implemented in the current version to avoid compromising forensic validity. In future enhancements, caching solutions such as Redis may be introduced to store recently processed metadata. Any caching implementation must include proper invalidation strategies to ensure that changes in files are accurately reflected during analysis.

Overall, the system prioritizes accuracy and reliability over performance optimization in its current design.

10. Non-Functional Requirements

10.1 Security Aspects

Security is a critical requirement in the MedTrace Forensics system due to the sensitive nature of healthcare data being processed. The system is designed to ensure that only authorized users can access the platform and that all forensic data remains protected from unauthorized access or tampering.The following table describes the key security-related non-functional requirements of the system:

ID	Requirement	Description
NFR1	Authentication	All system routes are protected using session-based authentication.
NFR2	Credential Handling	User credentials are validated securely on the server side before granting access.
NFR3	Session Security	Sessions are managed using Flask SECRET_KEY to prevent tampering and misuse.
NFR4	Input Validation	User inputs such as directory paths are validated to prevent malicious attacks.
NFR5	Data Integrity	SHA-256 hashing ensures that files are not altered after scanning.
NFR6	Secure Communication	System supports HTTPS deployment using TLS/SSL for secure data transmission.
NFR7	Access Control	Unauthorized users are restricted from accessing protected resources.
NFR8	Error Handling	System handles invalid inputs and security errors without exposing sensitive data.

The system secures operations using authentication, validation, and data integrity checks to ensure reliable forensic analysis. It also minimizes exposure of sensitive healthcare data by focusing only on metadata, maintaining overall system security.

10.2 Performance Aspects

Performance plays an important role in ensuring that the MedTrace Forensics system can efficiently handle file scanning and analysis operations. The system is designed to provide reliable results while maintaining acceptable response times and resource usage.

The following table outlines the key performance-related non-functional requirements:

ID	Requirement	Description
NFR9	Scan Efficiency	System efficiently scans directories using optimized traversal methods.
NFR10	Memory Management	Files are processed in chunks to reduce memory consumption.
NFR11	Response Time	System provides timely results for small to medium datasets.
NFR12	UI Responsiveness	Charts and visualizations are rendered on the client side for smooth interaction.
NFR13	Report Generation	Reports are generated efficiently in CSV format during scan operations.
NFR14	Scalability	System supports handling larger datasets with future enhancements.
NFR15	Resource Utilization	Efficient use of CPU and memory during scanning and hashing operations.
NFR16	Browser Compatibility	Supports modern browsers such as Chrome, Firefox, and Edge.

11. References

The following references were used for designing and developing the MedTrace Forensics system:

S. No	Type	Title / Description	Source / Link
1	Framework	Flask Documentation	flask.palletsprojects.com
2	Python Library	hashlib Library	docs.python.org
3	Visualization	Chart.js Documentation	chartjs.org
4	Standard	NIST Digital Forensics Framework	nist.gov
5	Security	HIPAA Security Rule	hhs.gov/hipaa
6	Learning Resource	IBM Cybersecurity Resources	ibm.com/security
7	Academic	KTU Project Guidelines	KTU Resources
8	UI Resources	Google Fonts (Orbitron, JetBrains Mono)	fonts.google.com
9	Concepts	Digital Forensics Concepts	Standard textbooks
10	Tool	Visual Studio Code	code.visualstudio.com

These references supported the system design, implementation, and understanding of forensic and cybersecurity concepts.